Lucius 22040350
Assignment: Requirements & Agile Modelling – MerbanTech

Name: Lucius
Course: [DCIT 208]
Index Number: 22040350
Team Name: MerbanTech
Project: Document Processing Web Application for a Financial Institution

1. Distinguishing & Evaluating Requirements

1.1. Definitions & References

1. **User Requirements**
    - **Definition:** Simple descriptions of what users want the system to do, written in everyday language.
    - **Reference:** According to Sommerville (Ch. 4), user requirements explain "what" the system should do from the user's perspective. Pressman adds that these are needed to make sure everyone agrees on the goals before building anything.

2. **System Requirements**
    - **Definition:** More detailed instructions that explain the system's features and how it should work. They describe "how" the system will meet the user's needs.
    - **Reference:** Pressman (7.3) calls these “functional requirements” because they describe specific actions the system will perform. Sommerville sees them as the technical plans for designers and developers to follow.

3. **Non-Functional Requirements**
    - **Definition:** These are rules about the system's quality, like how fast it should be, how secure it is, or how easy it is to use.
    - **Reference:** In Chapter 5, Sommerville says these are very important for making sure users are happy with the system. He also notes they should be measurable (e.g., how fast the system responds).
4.

1.2. Evaluation Examples from MerbanTech

Requirement Type	Clarity & Completeness	Suggested Improvement
As a department user, I need a way to find and save client documents myself by searching with the account number, fund date, or department, so I don't have to ask IT for help.	**User (Story) Clarity:** This is a good user story because it clearly states who is using the system, what they want to do, and why they want to do it. **Completeness:** It's a complete requirement because it lists the exact filters (account number, fund date, department) and what the user	To make it easier to test, we should add **acceptance criteria**. For example: “When I use correct filters and click search, the system will show me the right results.” This follows Pressman's

	wants to achieve (getting documents on their own). Sommerville (Ch. 4) explains that user requirements should be simple and capture the user's goal, which this story does well.	advice to define what "success" looks like for each requirement (7.3).
After the system successfully scans a document, it must find the client's name, account number, fund date, and department, and then save that information to the database	**System (Func.) Clarity:** It clearly explains its job (getting data and saving it) and when it should happen (after the OCR scan is successful). **Completeness:** It lists all the data to be collected and where to save it. This follows Pressman's model for functional requirements, which says to describe the trigger, the action, and the result (7.3).	To make this better, we should explain what happens when there's an error. For example: “If the OCR scan is less than 90% sure, the document should be marked for a person to check.” This follows Sommerville’s tip to plan for errors and unusual situations (Ch 4).
The system must show search results in less than 2 seconds, even when it's searching through 100,000 documents.	**Non-Functional Clarity:** It sets a clear goal for speed (under 2 seconds) and the amount of data it can handle (100,000 records). **Completeness:** It mentions the conditions it needs to work ("normal network and hardware"). Sommerville notes that these kinds of requirements must be easy to measure and test (Ch 5).	To make it easier to test, we need to define what “normal network and hardware” means. For instance: “on a 1 Gbps local network and a server with 16 GB RAM and 8 vCPUs.” We should also name the tool for testing: “Measured using JMeter load tests.”

References

- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner’s Approach* (9th ed.), Chapter 7.3.
- Sommerville, I. (2015). *Software Engineering* (10th ed.), Chapters 4 & 5.

2. Drafting & Prioritising Requirements

2.1. Requirements for Bulk Upload & OCR

ID	Requirement	Type
R1	As a department user, I need to upload a big group of up to 500 PDFs at the same time, so I can get them all into the system quickly.	User
R2	The system should handle a ZIP file upload. It needs to check all the PDFs	Syste m

	inside and then get them ready for the next step.	
R3	The system grabs key details, renames the file, and saves the info. It must do this for every document in 30 seconds or less.	System
R4	To keep the user informed, the system must provide a live status update. This should track the total files and show how many have succeeded and how many have failed.	System
R5	The system must make a report file when it's done. This report needs to show the status of every file and list any problems.	System
R6	If a file fails to process, the system must automatically try again twice. If it still doesn't work after the third try, it should be set aside for manual review.	System
R7	To make troubleshooting easier, I, as an IT admin, want to download a report from the batch process. This will help me quickly identify and correct failed items without re-running the successful ones.	User

2.2. MoSCoW Prioritisation

Requirement(s)	Priority	Justification
R1, R2, R3	**Must**	To deliver any value, the system must include these key features: batch uploading, file validation, and OCR. They represent the absolute minimum needed for a working product, which is what

		Sommerville calls a 'must' requirement (Ch. 5).
R4, R5	**Should**	Adding these features boosts usability and transparency. They aren't critical for the initial release and can be postponed if we're short on time, but they should be next on our list.
R6	**Could**	Adding these features boosts usability and transparency. They aren't critical for the initial release and can be postponed if we're short on time, but they should be next on our list.
R7	**Won't**	While this feature is valuable for checking work, it's a lower priority. We will add it once the core system is stable and running.

2.3. Refinement Over Multiple Sprints

1. **Sprint 1 (2 weeks)**
    - **To Deliver:** R1, R2, R3 (We'll build the upload page, the system for queuing files, and OCR for one file at a time).
    - **Stakeholder Input:** We will show the basic upload feature to users to get their opinion on how it works and feels.
    - **Trade-offs:** We will leave out the progress bar and retry feature for now. The main goal is to make sure we can reliably process up to 100 files first.

2. **Sprint 2 (2 weeks)**
    - **To Refine:** Make R2 work for the full 500 files. We will also build R4 (the live progress bar) and R5 (the summary report).
    - **Stakeholder Input:** We will check the progress dashboard with the IT admins and ask them if they prefer the report in CSV or JSON format.
    - **Trade-offs:** If we are short on time, we will move the retry feature (R6) to the next sprint.

3. **Sprint 3 (2 weeks)**
    - **To Enhance:** Add R6 (automatic retries) and R7 (the downloadable report for IT admins).
    - **Stakeholder Input:** We will have IT admins test the new features to confirm the retry system works and the reports are correct.

- **Trade-offs:** We will limit the number of retries as agreed. Any files that still fail will be logged for someone to check manually, rather than building a new screen for reviews.

References: • Sommerville, I. (2015). Software Engineering (10th ed.), Chapter 5 (Requirements Prioritisation). • Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.), Chapter 7.3 (Functional Requirements).

3. Agile Process Models

Here I will compare Scrum and Kanban and suggest how we can use them for our MerbanTech project.

3.1. Scrum

- **Values & Principles:** Based on the idea of checking your work and making improvements regularly. It uses **teams with different skills** who work together daily. Work is done in **short cycles called sprints** (usually 2–4 weeks long).
- **Strengths:** It creates a **steady rhythm** for getting work done and getting feedback (Sommerville Ch 9). Everyone has a **clear role**, like the Product Owner who organizes the work and the Scrum Master who helps the team (Pressman Ch 16). Sprint goals keep the team **focused**.
- **Weaknesses:** The meetings (planning, reviews) can **take up a lot of time**. The fixed sprints can feel **too strict** if urgent work comes up. It also **depends on key people**; if the Product Owner is not available, it can slow things down.
- **Adaptation for MerbanTech:** We can use **2-week sprints** to match our school schedule. Team members can take turns leading the Sprint Review to get everyone involved. We can use GitHub Projects for **simple planning** to reduce paperwork.

3.2. Kanban

- **Values & Principles:** Its main idea is to **show the work process visually** on a board. It suggests **limiting how many tasks** you work on at once to stay focused. It's all about **delivering finished work** as soon as it's ready, without fixed sprints (Sommerville Ch 10).
- **Strengths:** It's very **flexible**, so new important tasks can be started right away. Limiting work helps find and fix slowdowns. It has **less overhead** because there are fewer required meetings.
- **Weaknesses:** It's **harder to predict** when work will be finished without set deadlines. It can be easy to **keep adding new work** without a clear end point. The team must be **disciplined** about updating the board.
- **Adaptation for MerbanTech:** We can set **limits on work in progress** on our GitHub board (for example, no more than 3 tasks being worked on at once). We can have **weekly check-ins** instead of formal meetings. We can also **color-code tasks** to see what type of work we are doing.

3.3. Hybrid Approach

A mix of both, called **Scrumban**, seems like the best choice for us, since we have school schedules and client deadlines.

- **Planning Cadence:** We'll do simple sprint planning every 2 weeks.
- **Visual Board:** We'll use a Kanban board to see our tasks move from To Do to Done.
- **WIP Limits & Pull:** We'll limit ongoing tasks to avoid getting overwhelmed.

- **Retrospectives:** We'll have a short meeting at the end of each cycle to talk about what went well and what we can improve.
- **Continuous Delivery:** We can release new features as soon as they are ready instead of waiting for the end of a sprint.

This mix gives us the planning of Scrum and the flexibility of Kanban, which is perfect for balancing school and client work.

4. Process and System Modeling

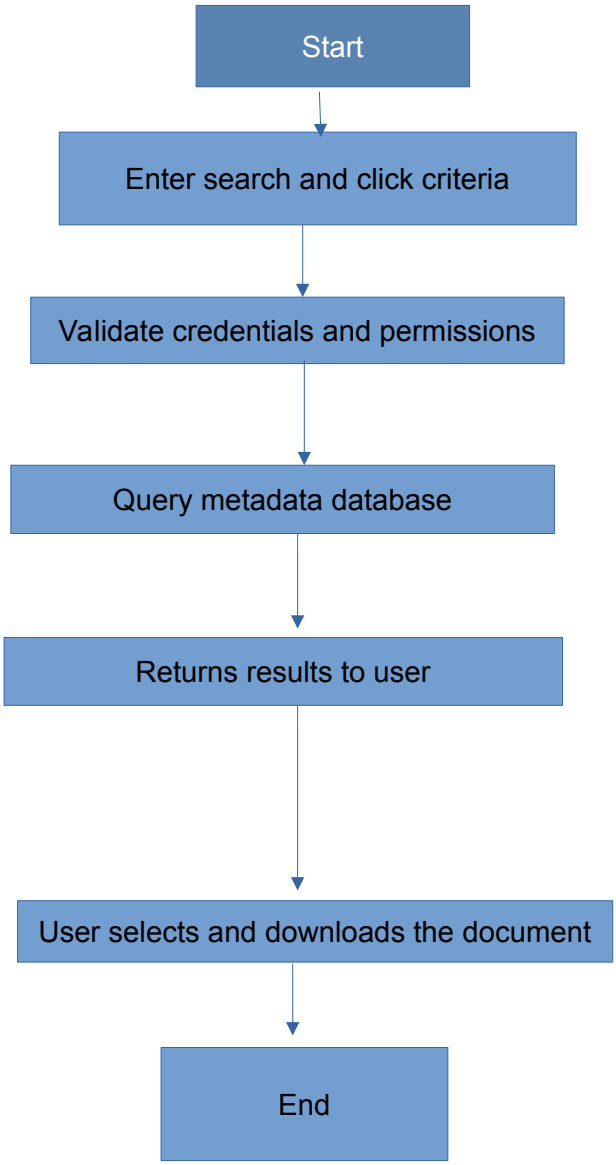

Figure 4.1 shows the whole process for the "Document Search & Retrieval" feature. Each step in the diagram matches our requirements and connects to a part of our system.

1. **Enter Search Criteria & Click “Search”**: This matches US-1 and R1, where a user types in filters on the website.
2. **Validate Credentials & Permissions**: Our security system handles this, making sure only the right users can move forward (as per US-4).

3. **Query Metadata DB**: This step uses the database to run searches, which meets the goals of US-1 and R2/R3.
4. **Return Results to User**: The system's API takes care of this, putting the data into a JSON package for the website, as required by R2.
5. **User Selects Document & Downloads**: This action is checked for security again before the file is sent to the user, meeting the needs of US-1 and our security rules.

This flow shows how our plan connects directly to the requirements and that each part of the system has a clear job.

Architectural Mapping:

- **UI Layer:** Steps 1 & 5 (the website)
- **Service Layer:** Steps 2 & 4 (the system's API)
- **Data Layer:** Step 3 (the database)
- **Security Layer:** Part of Steps 2 & 5 (for user login and permissions)

5. User Stories, Epics & Features

5.1. Definitions

- **Epic:** A big task that can be split into smaller ones.
- **Feature:** A feature that does something useful for the user.
- **User Story:** A short and simple request from a user's perspective, written to be clear and testable.

5.2. Epic, Features & User Stories

Epic E3: Bulk Document Upload & Processing
This epic is about letting users upload many PDFs at once and having the system process them automatically.

Features

1. **F1: Batch Upload Interface**
    - This is a screen where users can select and upload up to 500 files at the same time.
2.
3. **F2: Real-Time Batch Progress & Reporting**
    - This shows the live status of the upload and creates a downloadable report.
4.

User Stories

- **US-7 (for F1)**
  As a department user, I want to drag and drop many PDF files (up to 500) into an upload box *so that* I can start a large batch upload easily.
    - **Acceptance Criteria:**
        - The upload box lights up when I drag files over it.
        - The system only accepts PDF files and gives an error for wrong file types.
        - It shows me how many files I've selected before I click upload.
    -
-

- **US-8 (for F1)**
  As a department user, I want to see a confirmation message after selecting my files *so that* I can double-check the batch before starting.
    - **Acceptance Criteria:**
      - The message shows the file names and total count.
      - The “Confirm” and “Cancel” buttons work correctly.
      - Clicking “Cancel” removes the selected files.
    -
-
- **US-9 (for F2)**
  As an IT admin, I want to see a live progress bar for each batch *so that* I can watch the progress and spot any problems quickly.
    - **Acceptance Criteria:**
      - The progress bar moves as each file is finished.
      - It shows how many files succeeded and how many failed.
      - I can see error details if I hover over a failed item.
    -
-

5.3. Mini-Backlog Decomposition

Epic/Feature	Child Stories
E3: Document Upload in high volume	F1, F2
F1: Upload Interface	US-7, US-8
F2: Progress & Reporting	US-9

This table shows how our large Epic (E3) is broken down into two Features (F1, F2), which are then broken down into smaller User Stories. Each story is small enough to be completed in one sprint and clearly connects back to the main goal of the epic.